

Hackathon – Participant Support Document and Guidelines

Overview

The **MOCCM Black-Box Signal Hunt 2026** is designed to test the intersection of deep quantitative intuition and rigorous system engineering. Your team is tasked with moving away from generic, black-box machine learning APIs to build a transparent, ultra-low-latency mathematical trading infrastructure.

This document defines exactly what your code must accomplish, how your strategy phases are structured, and the precise formatting rules required to pass the automated grading infrastructure.

1. High-Level Strategy Architecture

Your team's technical workflow operates in two distinct functional phases. While your entire repository pipeline can live within a unified codebase, your architecture must cleanly separate these two operations:

Phase 1: The Quant Hunt (Signal Discovery)

Your first objective is to ingest and analyze the 10-year historical training dataset to isolate a hidden, non-linear, deterministic multi-asset relationship. You must identify how mathematical states, rolling thresholds, and directional variations in the 49 predictor assets dictate the forward price return of **TICKER_00** (dataset contains 50 tickers in total – target asset TICKER_00 and 49 other tickers, some of which predict the price of TICKER_00). This is your research phase where your mathematical and statistical theories are validated. ***If you make it to the Top 5, you will be expected to explain and defend your research in front of the judges.***

Phase 2: The Execution Engine (Backtest Engine)

Once your signal logic is discovered, you must code a mechanical loop that simulates a live trading desk. This engine must read market data sequentially, row-by-row, automatically evaluate whether your locked conditions are met at interval T , and execute a predictive trade affecting interval T to $T+1$.

On Sunday at 11:15 AM, when the completely unseen 5-year judging dataset is released, your team will have exactly 15 minutes to pipe that file directly through your **Phase 2 Execution Engine** to output your final production trade logs (the two csv files). The starting available cash is \$1,000,000 for the Long-Only strategy and \$2,000,000 for the Long-Short strategy. ***If you make it to the Top 5, you will also be expected to explain your portfolio constraint management / systems architecture / execution optimization logic (both long-only and long-short) in front of the judges.***

2. In-Memory Portfolio State Tracking

From the first line of data to the final timestamp, your engine must track and update four primary portfolio variables in-memory at every single 5-minute interval:

- **Cash_Balance:** Initiated at your total capital allocation tier. This ledger must adjust instantly whenever a transaction occurs.
- **Position_Shares:** The net number of units held in the target instrument (positive for long positions, negative for short positions).
- **Gross_Exposure:** The absolute market value of your open position, mapped mathematically as:

$$\text{Gross_Exposure} = |\text{Position_Shares} \times \text{Current_Price of TICKER_00}|$$

- **Gross_NAV (Net Asset Value):** The total pre-friction value of your fund at any specific interval, calculated via the core structural identity:

$$\text{Gross_NAV} = \text{Cash_Balance} + (\text{Position_Shares} \times \text{Current_Price of TICKER_00})$$

3. Operational Execution & Market Constraints

To survive the backend forensic accounting audit, your code engine must execute historical trades according to realistic institutional market mechanisms:

- **No Look-Ahead Bias:** Your calculations must evaluate data strictly available at or before interval T to make execution decisions for the next open interval. Ingestion of forward data points will trigger a structural disqualification.
- **Turnover Volume:** Whenever your engine adjusts its position size, it must compute the nominal dollar value traded during that specific 5-minute slice and log it to Interval_Turnover:

$$\text{Interval_Turnover} = |\text{Position_Shares}_T - \text{Position_Shares}_{T-1}| \times \text{Price}_T$$

- **Cash Flow Consistency:** Every row's change in cash must tie out perfectly with your transactional volume:

$$|\text{Cash_Balance}_T - \text{Cash_Balance}_{T-1}| = \text{Interval_Turnover}$$

- **Transaction Fee Drag:** The automated grader will automatically layer a **10 bps (0.0010) transaction cost** onto your cumulative Interval_Turnover to determine your final performance score. Your strategy must be robust enough to withstand this friction without wiping out your equity curve.

4. Risk Boundaries & Disqualification Gates



The automated grader enforces absolute risk parameters. Violating any of the following boundaries will result in an immediate score of zero:

- **The Flat Start/End Bounds:** Your strategy must start flat and end flat. Your Gross_Exposure must equal exactly 0.00 on the very first bar and the very last bar of the simulation window. All open positions must be completely liquidated before the final interval.
- **Strict Capital Ceilings:** Exposure limits are non-negotiable. Your Gross_Exposure can never cross **\$1,000,000** at any point in the Long-Only simulation, or **\$2,000,000** in the Long-Short simulation. Exceeding these limits by even one cent results in immediate strategy disqualification.
- **Margin Call Enforcement:** Your running Cash_Balance must stay positive. If your account cash dips below zero at any interval row, the grader will trip a margin call fatal error.

5. Final Output Schema Specifications

Outputs expected by 11AM on Sunday 7th June

- **teamname_code:** Your complete custom backtesting engine codebase.
- **teamname_proof.pdf:** Your written mathematical and statistical research proof explicitly detailing your discovered signal logic. This is the 10 mins presentation that you will use to present to the judges (if you are in the Top 5).

Outputs expected by 11:30AM on Sunday 7th June

- **teamname_longonly_results.csv:** The complete chronological 94,500-row execution log for your long-only strategy (capped at \$1,000,000 gross exposure).
- **teamname_longshort_results.csv:** The complete chronological 94,500-row execution log for your long-short strategy (capped at \$2,000,000 absolute gross exposure).

For your final output submission, your engine code will run on a 5-year judging dataset that will be provided separately after your code submission on Sunday (and not on the 10 year training dataset provided on Friday). The engine code is expected to output its complete execution history into two distinct, flat comma-separated files: **teamname_longonly_results.csv** and **teamname_longshort_results.csv** (*filenames should be entirely in lowercase format with no spaces*).

The Unbroken Timeline Rule

The output CSV is an **unbroken historical timeline**, not just a ledger of executed trades. It must contain exactly **94,500 consecutive rows** for the 5-year judging window, representing every single 5-minute market interval from start to finish.

Even if no trade occurs during a specific 5-minute window, a row must still exist. In those intervals, Interval_Turnover is logged as 0.0, while Gross_Exposure, Cash_Balance, and Gross_NAV carry forward to reflect the



current passive state of the portfolio. Skipping timestamps will trigger an immediate row-count failure.

Files must match the exact case-sensitive layout below:

Column Name	Expected Format	Technical Definition
Timestamp	YYYY-MM-DD HH:MM:SS	Chronological, monotonic market intervals.
Gross_Exposure	Float	Absolute nominal dollar value of the position.
Cash_Balance	Float	Running cash account balance following execution turnover.
Interval_Turnover	Float	Gross volume value traded during this specific 5-minute slice.
Gross_NAV	Float	Total asset value status (Cash_Balance + position market value).

Note: Use the validation checking script ([moccm_grader_modified.py](#)) provided in your repository setup to run structural sanity passes over your output data locally before the final drop window opens. Keep your output files in a folder named [submissions](#) and run the script.